Two Independent Monolingual Language Models: Telugu (Model H) and Nepali (Model L)

Data Collection, Transformer Implementation, Pretraining, and Reasoning Finetuning

Abhishek Reddy Kommuru

September 2026

Abstract

I built two decoder-only language models of about 25M parameters each, one for Telugu and one for Nepali, sharing no data, tokenizer, vocabulary or weights at any point. Telugu reached 639.9M training tokens with an 8,000-piece vocabulary and Nepali 497.0M with 32,000. Three results stand out. First, the two models are at parity on bits-per-byte (0.4588 against 0.4529) even though perplexity makes Telugu look 2.5 times better, because perplexity is measured per token and a token is a different sized object in each model. Second, finetuning on a synthetic comparative-reasoning set taught both models the output format but not the comparison: Telugu reaches 40.0% exact match against a 38.4% chance rate, and Nepali falls below chance. Third, removing positional embeddings costs only about 0.1 nats but visibly dismantles the attention structure the models had learned.

1 Introduction

The project asked for two monolingual language models built end to end from separate data, with one language chosen for its abundance of text and one for its scarcity. I picked Telugu as Model H and Nepali as Model L. The choice was not only about corpus size. Telugu occupies its own Unicode block, so deciding whether a document is Telugu is close to a solved problem. Nepali is written in Devanagari, which it shares with Hindi, Marathi, Maithili, Bhojpuri and Sanskrit, so the same question needs real work. That asymmetry turned out to shape the whole project, and it is the thread running through this report.

Nothing crosses between the two pipelines. Each language has its own source list, its own cleaning thresholds, its own tokenizer trained on its own training split, its own model config, its own weights, and its own finetuning data. Every stage script takes a language flag and reads only that language's files.

Table 1 is the whole project in one view. The rest of the report works through the three phases and then answers the four questions the assignment sets.

Table 1: Project summary. All figures are measured, not estimated: token counts come from the trained tokenizer, not from a bytes-per-token approximation.

	Telugu (Model H)	**Nepali (Model L)**
Training tokens / documents	639,877,839 / 1,097,873	497,000,869 / 3,157,122
Mean tokens per document	583	157
Manually collected share	15.26%	8.34%
Tokenizer (SentencePiece)	unigram, 8,000	unigram, 32,000
Bytes per token	9.92	12.97
Parameters / layers × heads	24,562,944 / 12×6	24,906,624 / 7×6
Embedding share of parameters	13.3%	50.1%
Pretraining steps (epochs)	20,700 (2.12)	16,000 (2.11)
Test perplexity	24.15	59.37
Test bits per byte	**0.4588**	**0.4529**
Reasoning exact match, finetuned	**40.00%**	**9.11%**

2 Phase 1: Data Collection, Cleaning and Tokenizers

2.1 Collection

Most of each corpus came from public datasets: FineWeb-2, Sangraha verified, Wikipedia, IndicCorp v2, and for Nepali also NepaliText. Each is recorded with its licence in the language's `sources.yaml`. The rest I collected myself, which the assignment requires, using a crawler I wrote, Tesseract OCR of scanned books from archive.org, and text extraction from government PDFs that already carry a text layer.

The crawler obeys robots.txt including any declared Crawl-delay, which for three Nepali hosts meant waiting 10 to 20 seconds between requests. It sends a descriptive User-Agent with a contact address rather than pretending to be a browser, rate-limits itself to roughly two requests per second per host, and skips any host declaring `Content-Signal: ai-train=no` by name. For Nepali that excluded `ekantipur.com` and `himalkhabar.com`, two of the largest archives available. I only publish extracted text and its provenance, never the raw HTML.

One detail cost me a lot of Nepali data before I found it. Python's `urllib.robotparser` sets `disallow_all` when fetching robots.txt returns HTTP 403, and bot protection on several Nepali sites returns 403 to the

datacenter IPs that Colab and Kaggle run on. That 403 is the shield talking, not the site's policy, and RFC 9309 §2.3.1.4 permits assuming no restrictions when robots.txt is unreachable. I now fetch robots.txt directly and parse it only on a 200 response. Two hosts that had produced almost nothing were affected.

Every document is stored with its source, licence, URL, fetch date, collection method and the list of filters it passed. That means the composition of the corpus can be recomputed from the data rather than taken on trust, which is how the manual-share figures in Table 2 were produced.

Table 2: Corpus composition. Source percentages are shares of training tokens.

	Telugu	**Nepali**
Manual tokens (own crawl, OCR, PDFs)	97,666,948 (15.26%)	41,458,012 (8.34%)
Downloaded tokens	542,210,891	455,542,857
Bytes of training text	6,346,125,017	6,444,456,393
Validation / test documents	15,985 / 16,252	41,224 / 41,061
Largest source / second	fineweb2 46.6% / sangraha 26.6%	**indiccorp_v2 36.4%** / fineweb2 32.5%

2.2 Splitting before cleaning

Validation and test, one percent each, are carved out before any cleaning touches the data. If you clean first and split afterwards, every threshold you chose has already seen the held-out data, and any filter that correlates with text quality has leaked information across the boundary. Splitting first keeps the held-out set an honest sample of what was collected. The assignment is deterministic in document-id order and proportional per source, so validation reflects the corpus mix rather than whichever source happened to be written first. The cost is that the held-out splits are uncleaned, which I accept, because the direction that matters for honest evaluation is covered by decontamination: stage 06 removes training documents that duplicate held-out text.

2.3 Cleaning, and why the two configs differ

Cleaning runs in four steps. Normalisation strips invisible characters, unifies whitespace, applies NFC, resolves the Devanagari nukta, removes zero-width joiners, maps native digits to ASCII and normalises punctuation while leaving the danda alone because it is a real sentence terminator. NFC runs twice, because the nukta and joiner steps can leave decomposed sequences behind. Language identification and quality filters follow, and finally email addresses and phone numbers are replaced with placeholders rather than deleted, since deleting a token mid-sentence leaves ungrammatical training text.

The two configurations are deliberately different, and this is the part of Phase 1 I thought hardest about. Telugu has data to spare, so it filters hard: a doubtful document costs nothing I cannot replace. Nepali does not, so it filters gently for exactly the opposite reason. Every relaxation is annotated with its reason in the file. Table 3 shows the important ones.

Two decisions are worth singling out. Telugu's mean-word-length ceiling is 12.0, raised from the 10 that English-tuned pipelines use, because I measured Telugu at 7.82 characters per word and the tighter bound was removing real text. And Nepali runs in repair mode: a document that fails only a repetition test has the offending lines deleted and is kept, rather than discarded.

Reading what the filters removed also caught a genuinely destructive rule. The terminal-punctuation test was removing roughly 70% of good Telugu articles, because a single-paragraph news article is one line, and if that line does not end in a full stop its ratio is zero. Applying line-structure rules only to documents of five lines or more took Telugu survival from 64.6% to 92.2%. I would not have found that without printing rejected documents and reading them.

Table 3: Selected cleaning thresholds. Each Nepali relaxation is annotated with its reason in `nepali/configs/clean.yaml`.

Threshold	**Telugu**	**Nepali**	**Reason for the difference**
Language-ID min confidence	0.70	0.50	GlotLID less sure on genuine Nepali
Min script ratio	0.85	0.80	Devanagari shared with five languages
Function-word discriminator	off	**on**	closes the gap the lower gate opens
Mean word length band	3.0–12.0	2.5–10.0	measured 7.82 vs 6.48 chars/word
Repair mode	off	**on**	repair rather than discard

2.4 Language identification

For Telugu a script-ratio test is almost the entire decision, since no other language uses the block. Nepali needed three gates. The first is script ratio, computed over letters only so that a table of numbers is not mistaken for the wrong language. The second is GlotLID, at a confidence threshold of 0.50 rather than Telugu's 0.70, because GlotLID is systematically less confident on real Nepali precisely when its Devanagari neighbours compete for probability mass; a confident vote for one of those neighbours is a hard reject. The third is a function-word discriminator that counts Nepali markers against Hindi ones and requires a smoothed ratio of at least 1.5. The

words it uses are Nepali copulas and case markers whose Hindi counterparts simply do not occur, so even a short document gives it enough signal. The neighbour check rejected 115 Maithili and 41 Hindi documents, which is evidence it fires on real contamination.

2.5 Deduplication

Exact duplicates are caught by hashing whitespace-collapsed text. Near duplicates need more. I represent each document as the set of its overlapping three-word shingles, so similarity becomes Jaccard overlap, compress each set to a 100-value MinHash signature, and use locality-sensitive hashing with 20 bands of 5 rows to avoid comparing most pairs at all. Two documents become candidates with probability $1 - (1 - s^r)^b$, which at these settings is 0.9996 for similarity 0.8 and 0.047 for 0.3, with the curve's steep point at $(1/20)^{1/5} \approx 0.55$. Candidates are then verified with a real Jaccard computation. Deduplication runs across all sources at once rather than per source, so a page my crawler fetched is matched against the copy sitting inside FineWeb-2, and where they collide the manually collected copy is kept.

Nepali received exact deduplication only. The MinHash pass did run and completed its clustering, measuring 10,844 exact and 38,218 near duplicates across 1,151,127 clusters, but the Colab session died during the shard-writing pass and the output was lost. With the deadline close I rebuilt the corpus using the exact-only path rather than spending another four and a half hours. The near-duplicate code is the same one applied in full to Telugu; only this run is missing, and roughly 3% near-duplicates therefore remain in the Nepali corpus.

2.6 Tokenizers

Both tokenizers are SentencePiece models trained in unigram mode on that language's training split only. SentencePiece is the library and unigram is the algorithm it ran; I also implemented BPE, WordPiece and Unigram from first principles in `common/tokenizer/scratch.py` with no tokenizer library, but those are two orders of magnitude slower and cannot process 500M tokens in reasonable time.

Unigram works by starting from a large candidate vocabulary, scoring each word's most likely segmentation with Viterbi, and repeatedly pruning the pieces that contribute least likelihood, never pruning single characters so that every string stays segmentable. Byte fallback is enabled, which is why the measured unknown-token rate is exactly zero for both languages. Digits are split individually, which I chose with Phase 3 numeric comparison in mind.

The vocabulary size was chosen by a sweep over algorithms and sizes, evaluated on held-out text, under a written rule: take the smallest vocabulary whose characters-per-token is within 5% of the best observed, subject to the embedding not exceeding half the Phase 2 parameter budget, and to the vocabulary fitting in uint16. Table 4 shows the sweep. Going from 16,000 to 32,000 improves characters per token by about 10% but pushes the embedding from 36.7% to 53.6% of the model, which fails the rule outright. The rule selected 8,000 for Telugu on the final corpus and 32,000 for Nepali. In hindsight 8,000 was pushed lower than ideal, and I return to that in the discussion.

Table 4: Tokenizer sweep on held-out text. The embedding share is the fraction of a 25M-parameter budget the token embedding would consume at $d_{\text{model}} = 384$.

Language	Algorithm	Vocab	Chars/token	Tokens/word	Embedding share
Telugu	unigram	16,000	4.319	1.810	36.7%
Telugu	bpe	16,000	4.300	1.819	36.7%
Telugu	unigram	32,000	4.772	1.639	53.6%
Telugu	bpe	32,000	4.777	1.637	53.6%
Nepali	unigram	32,000	4.920	1.310	53.6%

Shipped: Telugu unigram 8,000 and Nepali unigram 32,000. Unknown-token rate 0.000000 for both.

3 Phase 2: Transformer Implementation and Pretraining

3.1 Architecture

The forward pass is a token embedding plus a learned absolute positional embedding, dropout, a stack of pre-norm transformer blocks, a final LayerNorm, and a linear projection to the vocabulary tied to the input embedding. The implementation uses only `nn.Linear`, `nn.Embedding`, `nn.LayerNorm` and `nn.Dropout`. There is no `nn.Transformer`, no `nn.MultiheadAttention` and no `F.scaled_dot_product_attention` anywhere in the model path.

Attention is written out in full. Four linear projections produce queries, keys, values and the output projection. The model dimension is reshaped into six heads of 64 and the head axis is transposed in front of time so the matmuls batch per head. Scores are $QK^\top/\sqrt{d_{\text{head}}}$. The scaling matters because a dot product over 64 dimensions sums 64 roughly unit-variance products, so its standard deviation is about 8; scores that large saturate the softmax, one weight goes to nearly one and the gradient through it effectively vanishes. Dividing by $\sqrt{d_{\text{head}}}$, not $\sqrt{d_{\text{model}}}$, holds the variance near one because the sum runs over the head dimension.

The causal mask is an upper-triangular boolean built in the module and applied with $-\infty$ before the softmax

rather than by zeroing weights afterwards. That ordering is not cosmetic: zeroing after the softmax would leave the remaining weights unnormalised and would already have let the masked positions contribute to the denominator. It is registered as a non-persistent buffer so it moves to the GPU automatically but stays out of every checkpoint.

Weight tying reuses the input embedding as the output projection. It saves 3,072,000 parameters for Telugu and 12,288,000 for Nepali, and without it Model L could not have been built at this budget at all, since an untied seven-layer Nepali model would need about 37.2M parameters.

I fixed width and head count at 384 and 6 for both languages and let only depth vary. That was deliberate. Nepali's larger vocabulary costs four times as much in the embedding, and if width were also free the two models would differ in three ways at once and the comparison would be uninterpretable. Letting depth absorb the vocabulary difference leaves exactly one architectural variable. Table 5 gives the arithmetic.

Table 5: Parameter budget. Per block: attention $4(d^2+d) = 591{,}360$, feed-forward 1,181,568, two LayerNorms 1,536, total 1,774,464.

	Telugu		**Nepali**	
Component	**Parameters**	**Share**	**Parameters**	**Share**
Token embedding	3,072,000	12.5%	12,288,000	49.3%
Positional embedding	196,608	0.8%	196,608	0.8%
Transformer blocks	21,293,568	86.7%	12,421,248	49.9%
Final LayerNorm	768	0.0%	768	0.0%
Output head (tied)	0	—	0	—
Total	**24,562,944**		**24,906,624**	

3.2 Checks before spending GPU time

Four checks run before training. The parameter count is compared against the arithmetic derived from the config. The loss before any training is compared against $\ln V$, which is what a uniform distribution over the vocabulary gives: I measured 9.0796 against $\ln 8000 = 8.9872$ and 10.4386 against $\ln 32000 = 10.3735$. The causal mask is tested by altering the token at position $t+1$ and confirming that logits at positions up to t do not move at all, which they do not, while logits at and after it do, which confirms the perturbation was real rather than the test passing vacuously.

The fourth check exists because it caught a real bug. It trains a tiny model on the sequence $0, 1, 2, \ldots$ and asserts that it predicts the following token. My data layer already returned targets shifted by one, and an earlier version of `forward()` shifted them again, so the model was being trained to predict two positions ahead. Nothing else would have revealed this, because both alignments start at exactly $\ln V$ and both train downward. The loss curve would have looked completely normal while the model learned the wrong task.

3.3 Pretraining

Both models were trained with AdamW at $\beta = (0.9, 0.95)$, weight decay 0.1 applied only to tensors of two or more dimensions since decaying LayerNorm gains and biases pulls parameters that have no business shrinking, a 500-step linear warmup to 3×10^{-4} followed by cosine decay, gradient clipping at global norm 1.0 applied after unscaling, and gradient accumulation giving an effective batch of 65,536 tokens per step.

One practical finding is worth recording. Setting precision to `auto` selected bfloat16, which ran at 5.1 seconds per step. The T4 is a Turing GPU with no bfloat16 tensor cores, so bfloat16 is emulated at roughly fp32 speed. Forcing fp16 gave 1.39 seconds per step, 3.7 times faster, and that was the difference between 0.6 and 2.1 epochs within the same session budget. Gradient norms stayed between 0.5 and 0.6 throughout with no scaler collapse, so the narrower fp16 range cost nothing here.

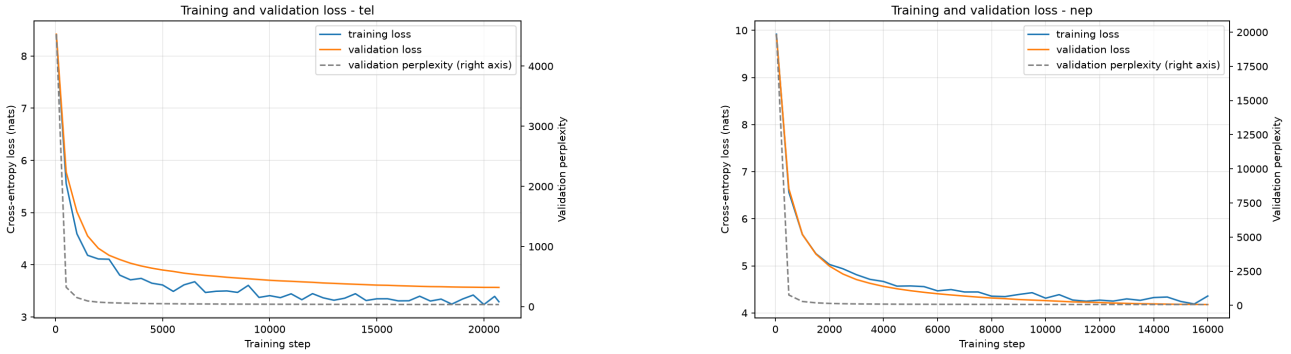
I matched epochs rather than wall clock: 2.12 for Telugu and 2.11 for Nepali. Nepali's faster step time would have allowed 3.59 epochs in the same hours, but then any difference in results could have come from the languages, the corpora, or simply from one model seeing its data 70% more often. Matching removes that variable. The Kaggle session died twice during the Telugu run, at steps 6,560 and 19,600, and resumable checkpoints meant losing at most 600 steps each time rather than starting over.

Figure 1 shows both curves. Training and validation stay close throughout, with a final gap of 0.27 nats for Telugu, so there is no meaningful overfitting even at 2.1 epochs. Both curves were still descending when the schedule ended, which says the models were limited by compute rather than by data. Nepali's training loss actually ends slightly above its validation loss, which looks wrong but is not: the training figure is a single noisy step while validation averages fifty batches, and dropout is active for one and disabled for the other.

3.4 Language-modelling results, and why perplexity misleads here

Table 6 gives the results over the complete splits. By perplexity Telugu looks two and a half times better. By bits-per-byte Nepali is slightly ahead. Both numbers are correct and the second is the one that means anything.

Perplexity is measured per token, and a token is not the same object in the two models. Telugu's 8,000-piece vocabulary packs 9.92 bytes into a token and Nepali's 32,000-piece vocabulary packs 12.97. Every Nepali



(a) Telugu **(b)** Nepali

Figure 1: Training and validation loss. Both are still descending at the end of the schedule.

prediction therefore covers more text and is chosen from four times as many options, so its per-token perplexity is inflated by construction. Comparing 24.15 against 59.37 measures the tokenizers, not the models. Bits-per-byte, computed as $\mathrm{CE}_{\mathrm{nats}} \times \text{tokens} \times \log_2 e / \text{bytes}$, normalises by the underlying UTF-8 text, which is the same unit in both languages.

The practical consequence is that Model L reaches parity on the fair measure despite 22% less data, roughly half the manual share, seven transformer layers against twelve, and half its parameters locked up in its embedding.

Bits-per-byte is not perfectly tokenizer-neutral either, and it is worth saying so. At initialisation, before either model has learned anything, Telugu's bits-per-byte is 1.3081 and Nepali's 1.1577, so the larger vocabulary starts about 11% ahead purely from geometry: quadrupling the vocabulary costs two extra bits per token but buys 31% more bytes. Measured as reduction from that starting point, Telugu achieved 64.9% and Nepali 60.9%. So Model L finishes ahead in absolute terms while Model H extracted slightly more from its data, which is closer to what the resource gap would predict.

Table 6: Language-modelling results, computed over the complete splits.

	Telugu val	**Telugu test**	**Nepali val**	**Nepali test**
Cross-entropy (nats/token)	3.1753	3.1843	4.0961	4.0838
Perplexity	23.93	**24.15**	60.11	**59.37**
Bits per byte	0.4579	**0.4588**	0.4541	**0.4529**
Tokens scored / bytes of text	9.32M / 93.2M	9.54M / 95.5M	7.21M / 93.8M	7.04M / 91.6M
Bits per byte at initialisation	1.3081		1.1577	
Reduction achieved	**64.9%**		60.9%	

3.5 Generation and attention

I sampled 200 held-out prefixes of 64 tokens, continued each for 64 tokens under greedy decoding and temperatures 0.5, 1.0 and 1.5, and scored them against the true continuation. The overlap metrics are close to uninformative in absolute terms: each compares against exactly one reference while language modelling is open-ended, so a model can produce excellent Telugu and score near zero. I report them because relative comparison across decoding settings is still meaningful.

The disagreement between them is the interesting part. BLEU peaks at temperature 0.5 (2.51) while chrF++ peaks at 1.0 (18.23). BLEU counts exact word n-grams and so rewards conservative decoding that reproduces common phrasings verbatim, while chrF++ works on character n-grams, so a correct stem with a wrong case ending still earns most of its credit. Telugu is agglutinative, where one wrong suffix destroys a word match while leaving most characters intact, so chrF++ is the more informative of the two here.

The real signal is in the diagnostics. Greedy decoding repeats 62.8% of its Telugu four-grams, and perplexity of 24.15 is completely blind to that. Raising temperature removes it monotonically, down to 0.0002 at temperature 1.0, while Distinct-1 climbs from 0.208 to 0.759.

For attention I measured per-head entropy and mean distance over 16 held-out sequences of 48 tokens, where uniform attention gives 2.931 nats. Both models learned the same three-stage organisation: diffuse aggregation, a sharp local middle band, then sharp long-range retrieval. Nepali reaches each stage two layers earlier, since with seven layers it cannot afford three diffuse ones at the bottom, and its long-range phase is two layers deep against Telugu's six with a shorter reach (12.43 against 16.01 positions).

The single result I find most striking is that both models independently learned a previous-token head: Telugu layer 5 head 2 with entropy 0.070 and mean distance 1.11, and Nepali layer 3 head 3 with entropy 0.276 and mean distance 1.11, identical to two decimal places, on different languages, corpora and tokenizers. Nothing in the architecture privileges that behaviour. The late-layer heatmaps also show the well-documented attention sink, a bright stripe at key position zero where heads with nothing useful to attend to park their softmax mass.

Table 7: Generation quality. 200 held-out prefixes of 64 tokens, each continued for 64 tokens and scored against the true continuation. BLEU-4 and chrF++ via sacreBLEU (`tokenize=intl`, `word_order=2`); ROUGE-L, Distinct-n and the repetition rate are computed in-repo.

	Decoding	BLEU-4	chrF++	ROUGE-L	Dist-1	Dist-2	4-gram rep.
Telugu	greedy	2.07	12.62	0.0496	0.208	0.306	**0.628**
	T=0.5	**2.51**	15.41	**0.0564**	0.396	0.652	0.215
	T=1.0	1.88	**18.23**	0.0403	0.713	0.987	0.0002
	T=1.5	0.98	16.93	0.0161	0.759	0.999	0.0000
Nepali	greedy	0.77	15.87	0.0621	0.113	0.181	**0.396**
	T=0.5	**1.01**	17.83	**0.0707**	0.300	0.618	0.123
	T=1.0	0.72	**19.22**	0.0618	0.533	0.947	0.0027
	T=1.5	0.08	17.45	0.0280	0.684	0.997	0.0000

It is a null operation rather than a bug, and every map is strictly lower-triangular, which is a visual confirmation of the causal-mask property verified numerically.

4 Phase 3: Reasoning Finetuning

4.1 The synthetic dataset

I generated the finetuning data programmatically so that the labels are known by construction. The generator draws the underlying structure first, either the entities and their numeric values or a hidden ordering, and renders sentences from it afterwards, so the answer is a consequence of the draw rather than a judgement about text. Nothing was downloaded, nothing translated, and no language model was involved. Every string a generated example can contain lives in the language's YAML config rather than in the code, so the phrasing can be corrected without touching the generator, and the generator is seeded so the dataset reproduces exactly.

There are five template families, listed in Table 8, running from a single comparison of two stated values up to a chain of three inequalities over four entities with no numbers given at all. The statements are shuffled inside every example, so the answer cannot be read off the surface order. That this works is measurable: always answering with the first-mentioned entity scores 26.9% for Telugu and 34.5% for Nepali, both below the 38.4% chance rate.

Leakage is prevented two independent ways, and the generator checks both rather than assuming them. The entity pools are disjoint across splits, so no name seen in finetuning appears at evaluation: 34 entities in training, 8 in validation and 12 in test. And `transitive_4` is generated for the test split only, so its 600 examples measure whether the model learned to compose inequalities or merely memorised the three-term surface form. Measured across all three split pairs, there are zero shared entities and zero repeated prompts.

Table 8: Reasoning dataset, identical in structure for both languages. `transitive_4` is held out of training entirely.

Family	Train	Val	Test	What it requires (hops)
`two_way_value`	6,000	400	800	compare two stated values (1)
`equality`	4,000	300	600	equal or not, answer yes/no (1)
`three_way_value`	6,000	400	800	maximum or minimum of three (2)
`transitive_3`	6,000	400	800	chain two inequalities, no numbers (2)
`transitive_4`	**0**	**0**	**600**	chain three inequalities, held out (3)
Total / distinct entities	**22,000** / 34	**1,500** / 8	**3,600** / 12	pools are disjoint

4.2 Finetuning protocol and what went wrong with it

Each model starts from its own pretrained checkpoint, keeps its own tokenizer and vocabulary unchanged, and sees only its own language's data. The hyperparameters are identical across the two languages by design, so that any difference in outcome is a property of the models rather than of the recipe: learning rate 3×10^{-5} decaying to 3×10^{-6}, an order of magnitude below pretraining because the model already speaks the language, 100 warmup steps, an effective batch of 64 examples, dropout reduced to 0.05, and loss computed on the answer tokens only. The prompt is given rather than predicted, and scoring it would let the model lower its loss by becoming fluent at restating the question. The prompt boundary is located by re-tokenising the prompt and verifying it is a prefix of the full encoding; across all 23,500 examples there were zero mismatches, so the masking is exact.

The configuration was badly over-budgeted, and this is the clearest mistake in Phase 3. Both models reached their best validation loss at step 100 of 1,500, and validation loss rose monotonically afterwards while training loss kept falling. Telugu went from 0.6334 at step 100 to 1.0896 at step 1,500 and Nepali from 2.5762 to 4.4350. A 25M-parameter model memorises 22,000 templated examples quickly, and 4.4 passes over them is far too many. The best-checkpoint mechanism saved the correct model in both cases, so the accuracy numbers reported below come from the step-100 checkpoints rather than the overfitted ones, but the right setting would have been roughly

200 steps evaluating every 25.

One number from this section already foreshadows the results: Telugu's best finetuning loss is 0.6334 and Nepali's is 2.5762, a factor of four on identical data with an identical recipe.

4.3 Reasoning results

I report two measurements. Exact-match generation gives the model the prompt, decodes greedily, and requires the output up to the first sentence terminator to equal the gold answer. That is the honest end-to-end test, and it is also unforgiving, because a pretrained model that has never seen the answer marker scores zero for reasons of format rather than reasoning. Likelihood-ranked choice scores each known candidate by its length-normalised log-likelihood and takes the best, which separates "cannot produce the format" from "does not know the answer".

Both are meaningless without a baseline. Guessing uniformly among each example's candidates gives 38.43%, because the families have two, three and four candidates respectively. Every number in Table 9 should be read against that, not against zero.

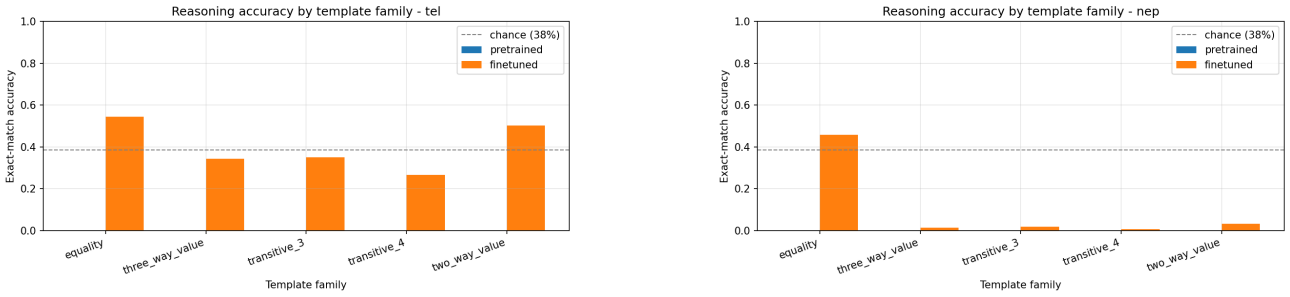
Read that way, the result is largely negative and I would rather say so plainly. Telugu's finetuned exact match of 40.00% is 1.6 points above chance. Nepali's 9.11% is 29 points below it. On four of five families Telugu lands within 1 to 4 points of chance, which is not a reasoning signal. The exception is `transitive_3`, where choice accuracy reaches 40.75% against a 33.3% chance rate and against its own pretrained score of 28.25%. That is the one family where something real was learned. Neither model generalised to the held-out four-entity chain.

The prediction distributions explain the gap between the two models better than any aggregate does. Telugu emits 28 distinct outputs across 3,600 test items, all valid whole entity names drawn from the correct test pool, with a single empty output. It learned which words are legal answers and then picks among them close to at random. Nepali emits 36 distinct outputs including a word-internal suffix fragment 827 times, a training-pool object that can never be a correct test answer 380 times, and 15 empty strings. That is a lexical competence failure rather than a reasoning one, and it is why Nepali's generation accuracy collapses on entity questions while its choice accuracy stays near chance: the ranking metric never requires it to produce the name.

Both models also solve `equality` with a constant. Telugu answers "no" for all 600 items and scores 54.33%, which is exactly the majority-class rate, and Nepali answers "yes" and scores 45.67%. Neither is comparing the two numbers. Finally, in both languages no example was broken by finetuning: it never turned a correct answer into a wrong one.

Table 9: Reasoning accuracy. Chance differs per family because the candidate count does. Other baselines: best constant answer 9.06%; first entity named 26.92% (Telugu) and 34.53% (Nepali).

	Telugu (Model H)			Nepali (Model L)		
Family	**Chance**	**Pre**	**Finetuned**	**Chance**	**Pre**	**Finetuned**
Exact match, greedy generation						
`two_way_value`	50.0%	0.0%	50.12%	50.0%	0.0%	3.25%
`equality`	50.0%	0.0%	54.33%	50.0%	0.0%	45.67%
`three_way_value`	33.3%	0.0%	34.25%	33.3%	0.0%	1.38%
`transitive_3`	33.3%	0.0%	34.88%	33.3%	0.0%	1.75%
`transitive_4` (held out)	25.0%	0.0%	26.67%	25.0%	0.0%	0.50%
Overall	**38.43%**	**0.00%**	**40.00%**	**38.43%**	**0.00%**	**9.11%**
Likelihood-ranked choice						
`transitive_3`	33.3%	28.25%	**40.75%**	33.3%	29.00%	28.25%
Overall	38.43%	35.61%	**41.75%**	38.43%	35.75%	35.11%



(a) Telugu **(b)** Nepali

Figure 2: Reasoning accuracy by family, pretrained against finetuned, with the chance line.

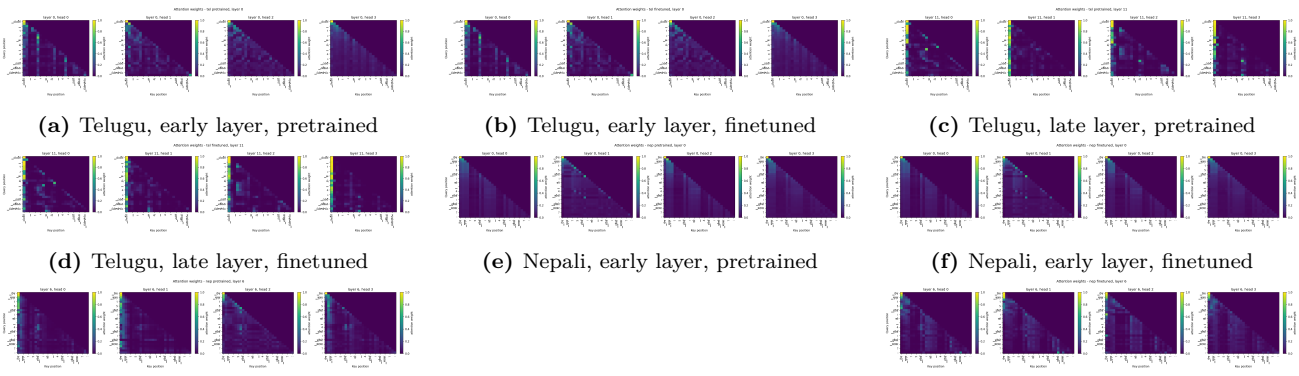
4.4 Attention after finetuning

I re-ran the Phase 2 attention toolkit unchanged over the same 16 comparative-reasoning prompts for both the pretrained and finetuned checkpoints, so the only variable is the weights. Using reasoning prompts rather than

corpus text is deliberate, since the question is whether finetuning changed how the model reads a comparison.

The two models moved in opposite directions, which I did not expect. Telugu's attention contracted: mean distance fell from 5.934 to 5.714 positions with 50 of 72 heads becoming more local, and the change concentrated in layers 3 to 5, which is exactly the local-sharpening band identified in Phase 2. Three of the six most-changed heads sit in layer 3 and all three sharpen and contract together. Nepali's attention expanded instead: mean distance rose from 6.281 to 6.834 with 29 of 42 heads reaching further, and its individual head changes are roughly twice the magnitude, with one head dropping a full nat of entropy while reaching 3.1 positions further.

Head specialisation is preserved rather than rebuilt. The largest single changes are concentrated in a few heads, L3H3 at −0.440 nats and L3H2 at −0.329 for Telugu, L4H2 at −1.019 and L5H5 at −1.005 for Nepali. The previous-token heads that Phase 2 found are not among them: measured on these prompts, Telugu's L5H2 moves from 0.178 to 0.209 nats and stays at 1.2 positions, and Nepali's L3H3 moves from 1.077 to 0.976 nats and stays at 1.9. Both remain previous-token heads after finetuning. So the task-specific reorganisation happens in other heads while the general positional machinery learned during pretraining is left intact.



(a) Telugu, early layer, pretrained **(b)** Telugu, early layer, finetuned **(c)** Telugu, late layer, pretrained

(d) Telugu, late layer, finetuned **(e)** Nepali, early layer, pretrained **(f)** Nepali, early layer, finetuned

(g) Nepali, late layer, pretrained **(h)** Nepali, late layer, finetuned

Figure 3: Attention on the same comparative-reasoning prompt, before and after finetuning, four heads per panel. Every map is strictly lower-triangular, which confirms visually what the causal-mask check verified numerically. The bright column at key position 0 in the late layers is the attention sink: softmax forces each head to distribute a total weight of one, so a head with nothing informative to attend to parks that mass at a fixed position. Full-resolution versions are in `report/figures/`.

A reading consistent with the accuracy results is that Model L, having seven layers to do work Telugu spreads over twelve, reorganises much more violently under the same pressure, and does so toward scanning the whole prompt rather than toward reading local entity-value pairs. Given that it never learned the answer vocabulary, that looks like a model searching its input rather than composing it. I would not push the interpretation further, since this rests on 16 prompts and on aggregate statistics over heads rather than a mechanistic account.

5 Bonus: Removing Positional Embeddings

I retrained both languages with the positional embedding table removed entirely rather than zeroed, costing 196,608 parameters, 0.8% of each model. Everything else was held fixed: same corpus, tokenizer, architecture, seed and fp16 precision. Because the ablated runs were given 6,000 steps against the standard models' 20,700 and 16,000, the honest comparison is at a matched step, which the recorded pretraining histories make possible.

At step 6,000 the cost of removing positions is 0.0937 nats for Telugu, a 9.8% increase in perplexity, and 0.1276 nats for Nepali, 13.6%. That is a much smaller penalty than the usual framing suggests. The permutation-invariance argument is correct about attention but incomplete about the model, because the causal mask is itself positional information: position t attends to exactly $t+1$ keys, so the prefix length differs at every position. What the model loses is the ability to say which of two visible tokens came first, and for next-token prediction a large share of the signal is local n-gram statistics that do not need absolute positions. The penalty is larger for Nepali, which is consistent with each of its tokens carrying 31% more text, so losing ordering destroys proportionally more structure.

What does break clearly is attention structure. Mean entropy rises from 1.8547 to 2.2216 nats for Telugu and mean distance falls from 11.03 to 9.11 positions, and the three-stage organisation is visibly degraded: the sharp local band at layer 3 disappears, its distance going from 4.82 to 9.97, and long-range reach at layer 10 falls from 16.01 to 12.09. Greedy four-gram repetition rises from 0.6283 to 0.6778 for Telugu and from 0.3964 to 0.5269 for Nepali, a third higher, which is intuitive for a model that cannot tell where it is in a sequence. The generation metrics separate two kinds of damage. Greedy chrF++ falls 9.4% for Telugu and 9.1% for Nepali while BLEU-4 barely moves in either. chrF++ counts character n-grams and BLEU word n-grams, so removing positions degrades character-level fidelity and provokes degeneration without changing how often whole words coincide with the reference. The two languages agreeing this closely on independently computed metrics is itself evidence the effect is real.

The same effect is visible directly in `report/figures/attention_late_tel.png` against `attention_late_tel_nopos.png`

the ablated model's heads are flatter across the whole map. The conclusion I draw is that this is not a story about capacity, since 0.8% of parameters is negligible, but about whether a class of attention head is representable at all. Phase 2 found both models independently learning a previous-token head with mean distance 1.11, an operation that is definitionally positional. Without a position signal no such head can exist, and the whole stack stays diffuse.

Table 10: Ablation results. The matched-step rows are the fair comparison; the final-checkpoint rows mix the ablation with a shorter training budget.

	Telugu		**Nepali**	
	Standard	**No positions**	**Standard**	**No positions**
Parameters	24,562,944	24,366,336	24,906,624	24,710,016
Matched at step 6,000				
Validation loss (nats)	3.8393	**3.9330**	4.4071	**4.5347**
Cost of removing positions	+0.094 **nats,** +9.8% **ppl**		+0.128 **nats,** +13.6% **ppl**	
Final checkpoints, unequal budgets				
Test bits per byte	0.4588	0.5095	0.4529	0.4913
Greedy BLEU-4	2.07	2.01	0.77	0.80
Greedy chrF++	12.62	**11.43**	15.87	**14.42**
Greedy 4-gram repetition	0.6283	**0.6778**	0.3964	**0.5269**
Mean attention entropy	1.8547	**2.2216**	2.0100	2.1612
Mean attention distance	11.033	**9.109**	9.204	8.452

6 Discussion

6.1 How data scale and quality differed

The scale gap is 22% in tokens, and it turned out to be the smaller part of the story. The manual share differed more, 15.26% against 8.34%, and both fell short of the 20% target. For Telugu that was a sizing decision: holding the floor would have capped the corpus near 390M, well under the target, so the code enforces the floor only as far as reaching the target allows and never downsamples manual data. For Nepali it was a collection failure, with four of seven hosts under-delivering and two of the largest archives excluded because they declare that they do not wish to be used for training.

Quality differed far more than scale, in three ways. Language identification was nearly free for Telugu and genuinely hard for Nepali, needing a three-gate ensemble with looser thresholds that necessarily admit more near-miss material. Deduplication was not equivalent, since Nepali received exact matching only and roughly 3% near-duplicates remain. And a substantial part of the Nepali corpus is word-level scrambled text, which I found in Phase 2 by reading held-out reference continuations: real Nepali words in correct Devanagari that do not form sentences. It passes every filter legitimately, because it is the right language in the right script with a normal symbol ratio, and scrambling actually reduces n-gram repetition, so the repetition filters see it as cleaner than natural text.

Two corpus statistics support this. Nepali has 3,157,122 documents averaging 157 tokens against Telugu's 1,097,873 averaging 583, a 3.7-fold difference in document length. And Nepali's largest single source is IndicCorp v2 at 36.4% of training tokens, which is one of the two candidates for a token-level rather than document-level dataset.

6.2 How language-modelling and reasoning results compare

They point in opposite directions, and that contrast is the most interesting outcome of the project. On language modelling the two models are at parity, with Nepali 1.3% ahead on bits-per-byte despite every resource disadvantage. On reasoning the gap is wide and favours Telugu: 40.00% against 9.11% exact match, one family above chance against none, and a finetuning loss four times lower.

The two tasks probe different things. Bits-per-byte rewards predicting the next token of whatever distribution the model was trained on, and if part of that distribution is word salad then a model that predicts word salad well still scores well. Reasoning requires producing a specific short answer from a constrained vocabulary in a structured format, which demands that the model has learned what a well-formed noun looks like as a complete unit. Nepali's fragment outputs show that it has not. Phase 2 could document the corpus problem but could not show that it mattered, because bits-per-byte is partly blind to it. Phase 3 is the evidence that it costs something.

6.3 Which factors most affected the lower-resource model

In order of impact, first the corpus destructuring, which removes exactly what a reasoning task needs, namely the association between a complete word form and its role in a sentence. Second the 32,000-piece vocabulary acting through the architecture: it costs 12.29M parameters in the embedding against Telugu's 3.07M, leaving only enough budget for seven transformer layers instead of twelve, with the attention analysis showing a long-range phase two layers deep instead of six and a shorter maximum reach. Third the incomplete deduplication, and fourth the looser language-identification gates, which trade precision for recall deliberately.

One factor mattered less than expected: raw token count. A 22% data gap barely registers when both models were still improving at the end of their schedules and neither had extracted what was already available.

6.4 What evidence explains the differences

The most direct evidence is behavioural. The prediction distributions show Telugu emitting valid whole entity names from the correct pool while Nepali emits suffix fragments, a training-pool word and empty strings, which is a lexical competence difference rather than a reasoning one. Next is the finetuning loss gap of four times on identical data with an identical recipe. Corroborating that are the incoherent Nepali held-out samples from Phase 2, and the opposite directions the two models' attention took under identical finetuning pressure. The ablation adds a structural data point, costing Nepali more than Telugu in a way consistent with its higher bytes-per-token.

One piece of evidence constrains the conclusion rather than supporting it. Bits-per-byte parity means Model L is not simply a worse language model, so whatever is wrong is specific to producing structured, lexically well-formed answers. And the direction of the scrambled-text bias cannot be settled from these measurements: shuffled text destroys long-range structure, which ought to make modelling harder rather than easier, so the naive expectation is that it hurts Nepali's bits-per-byte rather than flattering it.

7 Limitations and Reflection

The reasoning result is largely negative. Neither model learned comparative reasoning, and presenting 40% without its 38.4% baseline would have been misleading. Finetuning was over-budgeted by roughly fifteen times, and although the best-checkpoint mechanism saved the right model I never ran a proper sweep around the actual optimum. The ablation ran 6,000 steps against the standard models' full budgets, handled by the matched-step comparison but leaving the final-checkpoint table confounded. Nepali never received MinHash deduplication. Both manual shares fell below the 20% target. The reasoning templates were written by someone who is not a fluent speaker of either language, which is why every string sits in a config file where a speaker could correct it. And the attention comparisons rest on 16 prompts.

If I were starting again, the first thing I would do is find and drop the scrambled Nepali source, because almost everything else about Model L is downstream of it. I would finetune for about 200 steps rather than 1,500, and scale the reasoning dataset up so that a 25M model cannot memorise it in one pass. I would reconsider Telugu's 8,000-piece vocabulary, since the selection rule pushed it below the range the assignment suggested and a larger vocabulary would make the two models more comparable. I would train both models longer, since the session limit rather than the data stopped them. And I would use rotary positional embeddings instead of learned absolute ones, both to remove the hard 512-token ceiling and because the ablation showed how much of the attention structure depends on having a position signal at all.

The most useful habit I picked up was checking things before trusting them. The target-alignment test caught a bug that no loss curve would have shown, reading rejected documents caught a filter destroying 70% of good articles, and reading generation samples caught a corpus problem that every automated filter had passed.

Artifacts

Repository, branch `phase-3`:
`https://github.com/CL3-410/individual-project-Abhishekreddykommuru/tree/phase-3`
Google Drive, public viewing enabled on every item:

- Corpora and tokenized arrays:
  `https://drive.google.com/drive/folders/1AJO92VfuIbdJmp5GSaVmJz8BzUPa31I_`
- Pretrained `tel_best.pt`: `https://drive.google.com/file/d/1d6-hMx5m91LojrqwT1YYgCW4uuVxcWL_`
- Pretrained `nep_best.pt`: `https://drive.google.com/file/d/1enzNOOGkLtPIlS1a1HQZGt5TZrJcDP4w`
- Finetuned `tel_ft_best.pt`: `https://drive.google.com/file/d/1T1cS79d2VnK29_xLYMo2Hui2KLYE7gOx`
- Finetuned `nep_ft_best.pt`: `https://drive.google.com/file/d/1gOvLtRstiLDbHCoHBOqfLXG4J4DnHzfS`
- Ablated `tel_nopos_best.pt`: `https://drive.google.com/file/d/18BPIhTI-GnmYrpQ0pp0ZpdTvhxY9Nzbo`
- Ablated `nep_nopos_best.pt`: `https://drive.google.com/file/d/1N1oyV1WHniLPM-83egJVdShK3XSMfdGR`

Reproduction. `run_all.py`, `run_phase2.py`, `run_phase3.py --ablation`. All stages resume from checkpoints and the reasoning generator is seeded, so the dataset reproduces byte for byte. No pretrained models, tokenizers or HuggingFace transformers were used; attention, positional embeddings and causal masking are in `common/model/`.